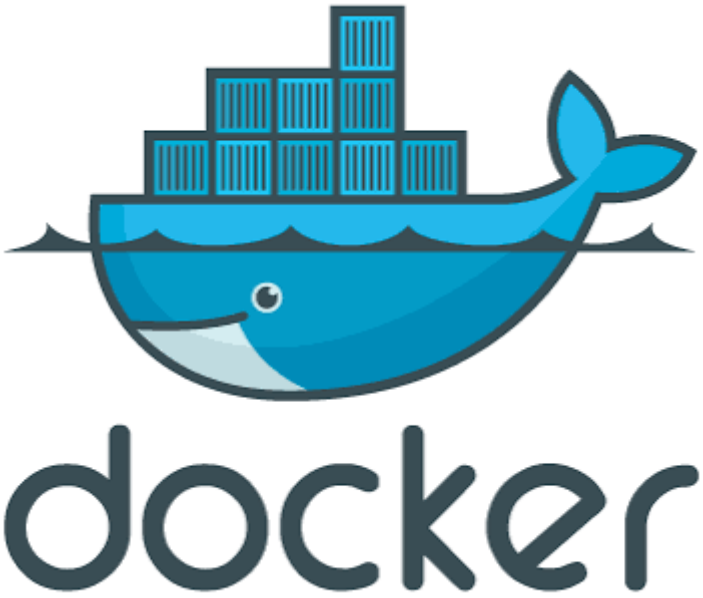


UDO1_DockerNintendo64

Alumno: Manuel Jesús Jiménez Romero.
Curso: 2º Desarrollo de aplicaciones Web(26/27)

1:Juegos de Nintendo en un docker. **2**
Parte 2: Esta configuración se le añade a docker-compose.yml **3**

Manuel Jesus Jimenez Romero
UDO1_DockerNintendo64

Parte 1:Juegos de Nintendo en un docker.

Se crea el directorio emulador_nintendo, luego nos situamos en ese directorio y dentro de este se crea el directorio config y data.

```
PS C:\Users\Manuel J> mkdir emulador_nintendo
>> cd emulador_nintendo
>> mkdir config
>> mkdir data
```

```
    Directorio: C:\Users\Manuel J\emulador_nintendo


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        24/10/2026     13:54                config
d-----        24/10/2026     13:54                data
```

---

creamos assets, library\rows y un yml llamado docker-compose.yml

```
PS C:\Users\Manuel J\emulador_nintendo>
>> mkdir library\roms
>> mkdir assets
```

```
    Directorio: C:\Users\Manuel J\emulador_nintendo


Mode                 LastWriteTime         Length Name
d-----        24/10/2026     14:03                assets
d-----        24/10/2026     13:54                config
d-----        24/10/2026     13:54                data
d-----        24/10/2026     14:03                library
-a----        24/10/2026     14:02            374 docker-compose.yml
```

```
PS C:\Users\Manuel J\emulador_nintendo> -join ((1..32) | ForEach-Object { '{0:x2}'
-f (Get-Random -Maximum 256) })
65dc5e683b06c20e3a3da9086862440d0c9c28cd4d4719835c28e82198e11bfd
```

Se añade este comando para generar la clave secreta.

Parte 2: Configuración

Esta configuración se le añade a docker-compose.yml

```
services:
  romm:
    image: rommapp/romm:latest
    container_name: romm
    restart: unless-stopped
    environment:
      - DB_HOST=romm-db
      - DB_NAME=romm
      - DB_USER=romm-user
      - DB_PASSWD=tumorenito17   # debe coincidir con MARIADB_PASSWORD
      - ROMM_AUTH_SECRET_KEY=PEGA_AQUI_LA_CLAVE_GENERADA
      - HASHEOUS_API_ENABLED=true
    volumes:
      - romm_resources:/romm/resources
      - romm_redis_data:/redis-data
      - ./library:/romm/library
      - ./assets:/romm/assets
      - ./config:/romm/config
      - "C:/Users/Manuel J/Downloads/Minerva_Myrient/No-Intro/Nintendo - Nintendo 64
(BigEndian):/romm/library/roms/n64:ro"
    ports:
      - "8086:8080"
    depends_on:
      romm-db:
        condition: service_healthy
        restart: true

  romm-db:
    image: mariadb:latest
    container_name: romm-db
    restart: unless-stopped
    environment:
      - MARIADB_ROOT_PASSWORD=tumorenito17
      - MARIADB_DATABASE=romm
      - MARIADB_USER=romm-user
```

```
    - MARIADB_PASSWORD=tumorenito17   # debe coincidir con DB_PASSWD
  volumes:
    - mysql_data:/var/lib/mysql
  healthcheck:
    test: [CMD, healthcheck.sh, --connect, --innodb_initialized]
    start_period: 30s
    start_interval: 10s
    interval: 10s
    timeout: 5s
    retries: 5

volumes:
  romm_resources:
  romm_redis_data:
  mysql_data:
```

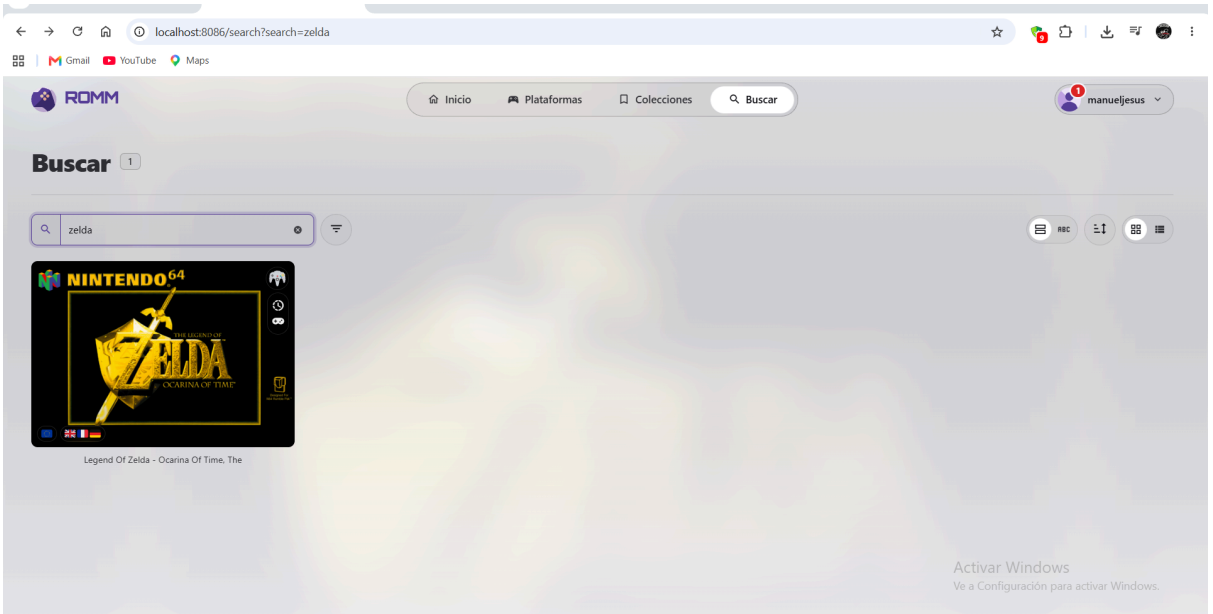
Parte 3: Final y Comprobación

Luego de modificar el yml se guardara, luego se activa y se comprueba que lea cada rom de la carpeta nintendo.

```
PS C:\Users\Manuel J\emulador_nintendo> docker compose config
```

```
docker compose up -d
```

Nos metemos al localhost asignado el mio es 8086 para probar que funcionaba me descarge un rom del zelda.



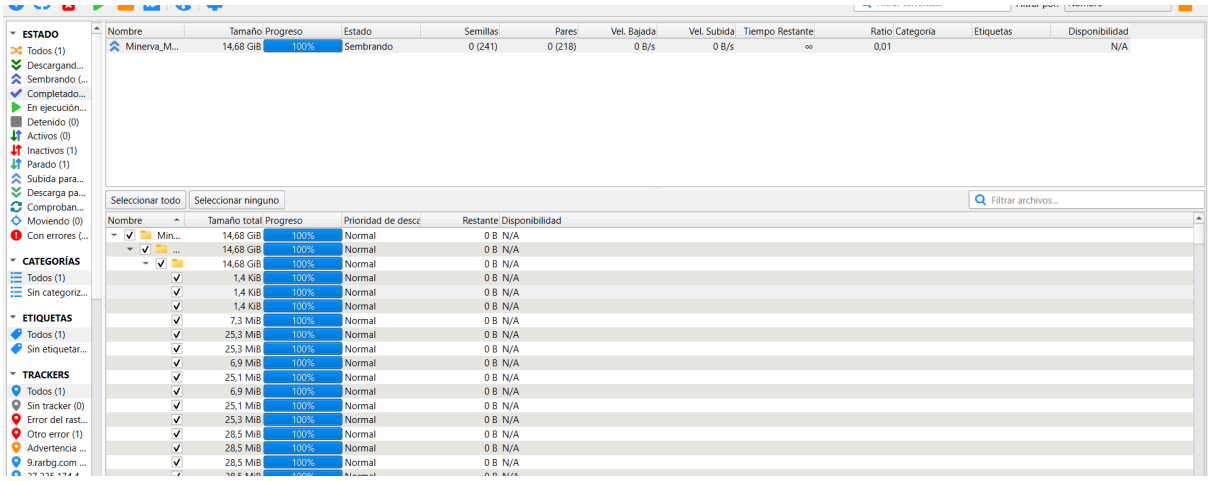
Manuel Jesus Jimenez Romero
UDO1_DockerNintendo64

Parte 4: Archivo de Nintendo

Luego vamos a descargar torrent para descomprimir archivos grandes y nos iremos a minerva donde descargaremos el .zip donde estan todos los juegos. Descargamos el de .torrent



Con esto ahora lo llevaremos a qbtorrent donde nos dara cada zip que contiene cada juego



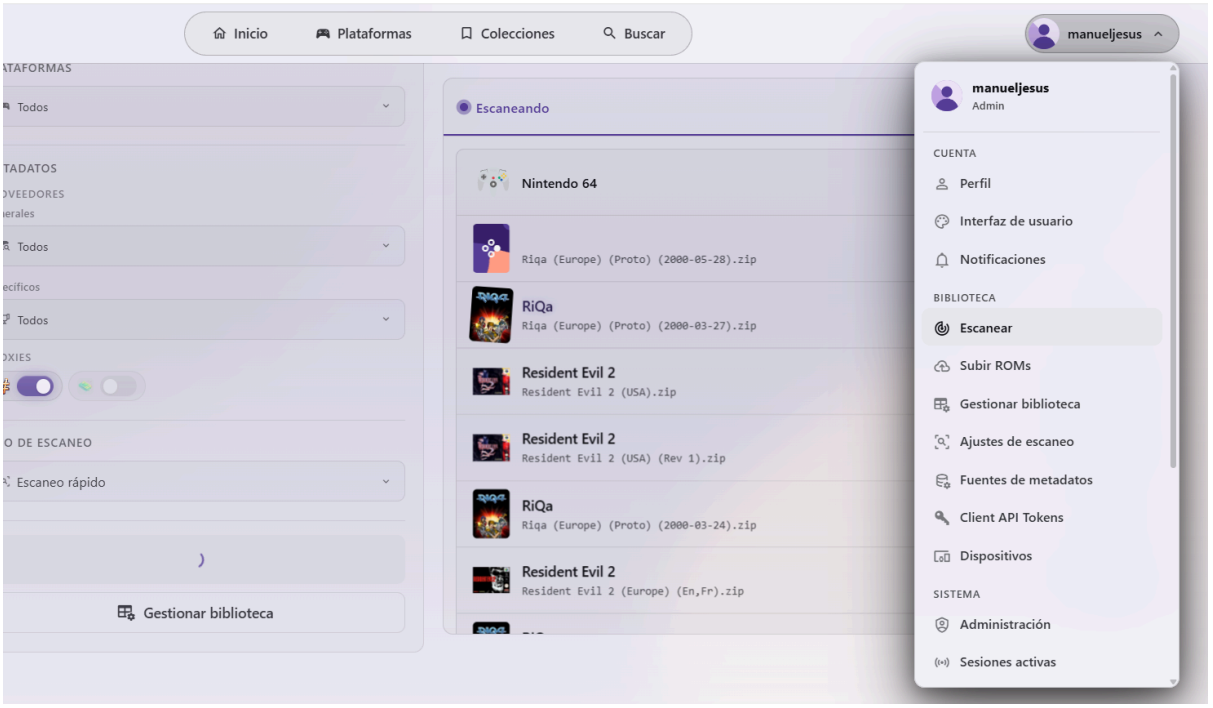
con esto ahora nos iremos a docker donde añadiremos el siguiente comando para ver si funciona correctamente.

```
PS C:\Users\Manuel J\emulador_nintendo> docker compose config
>> docker compose up -d
>> docker compose exec romm ls /romm/library/roms/n64
```

esto sirve para ver si lee cada rom como lo hace correctamente el docker esta correctamente configurado para leer cada romm y analizar el equipo en busca de ellos

```
Zelda no Densetsu - Toki no Ocarina (Japan) (GameCube, Zelda Collection).zip
Zelda no Densetsu - Toki no Ocarina (Japan) (Rev 1).zip
Zelda no Densetsu - Toki no Ocarina (Japan) (Rev 2).zip
Zelda no Densetsu - Toki no Ocarina (Japan).zip
Zelda no Densetsu - Toki no Ocarina GC (Japan) (GameCube).zip
Zelda no Densetsu - Toki no Ocarina GC Ura (Japan) (GameCube).zip
Zool - Majuu Tsukai Densetsu (Japan).zip
[BIOS] Nintendo 64 - PIF (Brazil).zip
[BIOS] Nintendo 64 - PIF (Europe).zip
[BIOS] Nintendo 64 - PIF (Japan, USA).zip
PS C:\Users\Manuel J\emulador_nintendo>
```

para analizar iremos a este apartado donde saldra analizar y cuando se le de empezara a buscar y saldra cada juego que tengamos que pueda reconocerlo



Manuel Jesus Jimenez Romero
UDO1_DockerNintendo64

